Chitkara

PAPER OF CSE-2025 - PYTHON TEST6 EXAM

Q1. Cookie Jar Safety

A row of cookie jars is kept on a kitchen shelf.

Each jar has a **weight**.

To stop the shelf from falling, the cook checks every **3 jars next to each other**.

If the **total weight** of those 3 jars is **more than 60**, the **middle jar** is taken away.

This keeps happening until no group of 3 jars is too heavy.

Your job is to return the **final list of jar weights**.

Rules :

- Look at every group of 3 jars.
- If their total weight is more than 60, remove the middle jar.
- Keep repeating until the list is safe.
- Finally, return the safe list of jar weights.

Input Format:

A list of integers showing the weight of each cookie jar.

Output Format:

A list of integers showing the safe jar weights.

Sample Test Case 1:

Input:

```
[20, 30, 15]
```

Output:

```
[20, 15]
```

Sample Test Case 2:

Input:

```
[10, 40, 25, 20]
```

Output:

```
[10, 25, 20]
```

ANSWER:

Test case 1 :

Input:
[20, 30, 15]
Output:
[20, 15]

Test case 2 :

Input:
[10, 20, 15, 10]
Output:
[10, 20, 15, 10]

Test case 3 :

Input:
[50, 15, 10, 20, 15]
Output:
[50, 15]

Test case 4 :

Input:
[15, 15, 15, 15, 15]
Output:
[15, 15, 15, 15, 15]

Test case 5 :

Input:
[30, 20, 15, 10]
Output:
[30, 15, 10]

Test case 6 :

Input:
[40, 25, 10]
Output:
[40, 10]

Q2. Apple Basket Safety

A row of apple baskets is kept on a table.

Each basket has a **weight**.

To stop the table from breaking, the helper checks every **3 baskets that are next to each other**.

If the **total weight** of those 3 baskets is **more than 60 kg**, the **middle basket** is removed.

This checking continues until **no group of 3 baskets** has a total weight above 60.

Your task is to return the **final list of basket weights** after everything becomes safe.

Rules :

- Look at 3 baskets that are next to each other.
- If the total is more than 60 kg, remove the middle basket.
- Start checking again from the beginning.
- Stop when no group of 3 baskets has a total over 60.
- Return the final list of basket weights.

Input Format:

A list of integers showing the weight of each apple basket.

Output Format:

A list of integers showing the final safe basket arrangement.

Sample Test Case 1:

Input:

```
[25, 20, 30]
```

Output:

```
[25, 30]
```

Sample Test Case 2:

Input:

```
[10, 40, 25, 20]
```

Output:

```
[10, 25, 20]
```

ANSWER:

Test case 1 :

Input:
[25, 20, 30]
Output:
[25, 30]

Test case 2 :

Input:
[10, 20, 15, 10]
Output:
[10, 20, 15, 10]

Test case 3 :

Input:
[50, 15, 10, 20, 15]
Output:
[50, 15]

Test case 4 :

Input:
[15, 15, 15, 15, 15]
Output:
[15, 15, 15, 15, 15]

Test case 5 :

Input:
[30, 20, 15, 10]
Output:
[30, 15, 10]

Test case 6 :

Input:
[40, 25, 10]
Output:
[40, 10]

Q3. Ice Cream Box Safety

A row of ice-cream boxes is kept on a freezer shelf.

Each box has a **weight**.

To prevent the shelf from breaking, the helper checks every **3 boxes that are next to each other**.

If the **total weight** of those 3 boxes is **more than 60 kg**, the **middle box** is taken off the shelf.

This checking continues until **no group of 3 boxes** is above the limit.

Your task is to return the **final safe list** of box weights.

Rules :

- Look at 3 boxes next to each other.
- If the total is more than 60, remove the middle box.

- Keep repeating until the list is safe.
- Finally, return the safe list of box weights.

Input Format:

A list of integers showing the weight of each ice-cream box.

Output Format:

A list of integers showing the safe box weights after checking.

Sample Test Case 1:

Input:

```
[15, 40, 10, 20]
```

Output:

```
[15, 10, 20]
```

Sample Test Case 2:

Input:

```
[10, 40, 25, 20]
```

Output:

```
[10, 25, 20]
```

ANSWER:

Test case 1 :

Input:
[15, 40, 10, 20]
Output:
[15, 10, 20]

Test case 2 :

Input:
[10, 20, 15, 10]
Output:
[10, 20, 15, 10]

Test case 3 :

Input:
[50, 15, 10, 20, 15]
Output:
[50, 15]

Test case 4 :

Input:
[15, 15, 15, 15, 15]
Output:
[15, 15, 15, 15, 15]

Test case 5 :

Input:
[30, 20, 15, 10]
Output:
[30, 15, 10]

Test case 6 :

Input:
[40, 25, 10]
Output:
[40, 10]

Q4. National Robotics League Robot Performance Summary

During the National Robotics League, each robot competes in multiple technical challenge rounds. Each challenge assigns a performance score based on speed, accuracy, and task completion efficiency.The competition committee stores these scores in a dictionary where the key is the robot's name and the value is a list containing its scores from various challenge stages.The committee now requires a structured report summarizing the performance of each robot.

Instruction/Task

Analyze the given robot performance data and generate a final evaluation summary.

For each robot:

1. Find Total Performance Score- Add all challenge scores.

2. Compute Average Mission Score → total_score / mission_count, rounded to 2 decimals.

3. Assign Robotics Grade

Use the following grading bracket:

A → Average ≥ 75

B → 50 ≤ Average < 75

C → Average < 50

4.Construct Output Dictionary: The result must map each robot to:

(total_score, average_score, grade)

Return this final summary dictionary.

Note:

While calculating the average mission score, **use the `round()` function** to round the value to **2 decimal places**.

Example:

```
average_score = round(total_score / mission_count, 2)
```

Input Format:

{RobotName : [score1, score2, score3, ...]}

Output Format:

{RobotName : (total_score, average_score, grade)}

Sample Input 1

```
{'BoltX':[90,75,82], 'IronBot':[45,55,50], 'NanoR':[60,62,58]}
```

Sample Output 1

```
{'BoltX': (247, 82.33, 'A'), 'IronBot': (150, 50.0, 'B'), 'NanoR': (180, 60.0, 'B')}
```

Sample Input 2

```
{'Alpha':[78,85,80], 'Beta':[35,40,45]}
```

Sample Output 2

```
{'Alpha': (243, 81.0, 'A'), 'Beta': (120, 40.0, 'C')}
```

ANSWER:

Test case 1 :

Input:
{'BoltX':[90,75,82], 'IronBot':[45,55,50], 'NanoR':[60,62,58]}
Output:
{'BoltX': (247, 82.33, 'A'), 'IronBot': (150, 50.0, 'B'), 'NanoR': (180, 60.0, 'B')}

Test case 2 :

Input:
{'Raj': [100, 100, 100]}

Output:
{'Raj': (300, 100.0, 'A')}

Test case 3 :

Input:

{'Ravi': [40, 45, 35], 'Tina': [60, 65, 70]}

Output:
{'Ravi': (120, 40.0, 'C'), 'Tina': (195, 65.0, 'B')}

Test case 4 :

Input:
{'Dev': [75, 75, 75], 'Simran': [49, 50, 51]}

Output:
{'Dev': (225, 75.0, 'A'), 'Simran': (150, 50.0, 'B')}

Test case 5 :

Input:
{'Anu': [100, 0, 100], 'Jay': [25, 25, 25]}

Output:
{'Anu': (200, 66.67, 'B'), 'Jay': (75, 25.0, 'C')}

Test case 6 :

Input:
{'Mira': [90, 80, 70], 'Kia': [55, 60, 65]}

Output:
{'Mira': (240, 80.0, 'A'), 'Kia': (180, 60.0, 'B')}

Q5. Online Gaming Arena Player Rank Generator

An online gaming platform tracks players' performance across multiple gaming missions.

Each mission gives a score, and these mission scores are stored in a dictionary with player names as keys.

The gaming system requires a performance ranking summary to assign reward tiers.

Instruction/Task

For each player:

1. Compute Total Mission Score → sum of all mission scores.

2. Compute Average Mission Score → total_score / mission_count, rounded to 2 decimals.

3.Assign Reward Rank:

- Rank 'A' → Average ≥ 75
- Rank 'B' → 50 ≤ Average < 75
- Rank 'C' → Average < 50

4.Prepare a new dictionary where each player maps to:

(total_mission_score, average_score, reward_rank)

Return this final reward summary.

Note:

While calculating the average score, **use the `round()` function** to round the value to **2 decimal places**.

Example:

```
average_score = round(total_score / number_of_rounds, 2)
```

Input Format:

Dictionary mapping:

{PlayerName : [mission_score1, mission_score2, ...]}

Output Format:

Dictionary mapping:

{PlayerName : (total, average, rank)}

Sample Input 1

```
{'Max':[70,80,75], 'Lina':[40,50,45]}
```

Sample Output 1

```
{'Max': (225, 75.0, 'A'), 'Lina': (135, 45.0, 'C')}
```

Sample Input 2

```
{'Rex':[85,90,88], 'Mini':[55,52,58]}
```

Sample Output 2

```
{'Rex': (263, 87.67, 'A'), 'Mini': (165, 55.0, 'B')}
```

ANSWER:

Test case 1 :

Input:
{'Max':[70,80,75], 'Lina':[40,50,45]}
Output:
{'Max': (225, 75.0, 'A'), 'Lina': (135, 45.0, 'C')}

Test case 2 :

Input:
{'Raj': [100, 100, 100]}

Output:
{'Raj': (300, 100.0, 'A')}

Test case 3 :

Input:
{'Ravi': [40, 45, 35], 'Tina': [60, 65, 70]}

Output:
{'Ravi': (120, 40.0, 'C'), 'Tina': (195, 65.0, 'B')}

Test case 4 :

Input:
{'Dev': [75, 75, 75], 'Simran': [49, 50, 51]}

Output:
{'Dev': (225, 75.0, 'A'), 'Simran': (150, 50.0, 'B')}

Test case 5 :

Input:
{'Anu': [100, 0, 100], 'Jay': [25, 25, 25]}

Output:
{'Anu': (200, 66.67, 'B'), 'Jay': (75, 25.0, 'C')}

Test case 6 :

Input:
{'Mira': [90, 80, 70], 'Kia': [55, 60, 65]}

Output:
{'Mira': (240, 80.0, 'A'), 'Kia': (180, 60.0, 'B')}

Q6. Corporate Fitness Challenge Employee Score Evaluator

A company arranged a “Fitness Challenge Week,” where employees participated in multiple fitness rounds. Each employee’s performance score in every round is stored in a dictionary. The management wants a structured analysis of overall fitness levels based on their cumulative performance.

Instruction/Task

Process the given employee fitness score record.

For each employee:

1. Compute Total Score → summation of all round scores.

2. Compute Average Score → total_score / number_of_rounds (rounded to 2 decimals).

3. Determine Fitness Category:

'A' → Avg ≥ 75

'B' → 50 ≤ Avg < 75

'C' → Avg < 50

4.Construct a results dictionary mapping each employee to:

(total_score, average_score, fitness_grade)

Return the final evaluation report.

Note:

While calculating the average score, **use the `round()` function** to round the value to **2 decimal places**.

Example:

```
average_score = round(total_score / number_of_rounds, 2)
```

Input Format:

Dictionary mapping:

{EmployeeName : [score1, score2, ...]}

Output Format:

Dictionary mapping:

{EmployeeName : (total, average, grade)}

Sample Input 1

```
{'Rohit':[60,75,80], 'Sameer':[40,45,50]}
```

Sample Output 1

```
{'Rohit': (215, 71.67, 'B'), 'Sameer': (135, 45.0, 'C')}
```

Sample Input 2

```
{'Divya':[90,85,95], 'Kritika':[55,60,50]}
```

Sample Output 2

```
{'Divya': (270, 90.0, 'A'), 'Kritika': (165, 55.0, 'B')}
```

ANSWER:

Test case 1 :

Input:
{'Rohit':[60,75,80], 'Sameer':[40,45,50]}
Output:
{'Rohit': (215, 71.67, 'B'), 'Sameer': (135, 45.0, 'C')}

Test case 2 :

Input:
{'Raj': [100, 100, 100]}

Output:
{'Raj': (300, 100.0, 'A')}

Test case 3 :

Input:
{'Ravi': [40, 45, 35], 'Tina': [60, 65, 70]}

Output:
{'Ravi': (120, 40.0, 'C'), 'Tina': (195, 65.0, 'B')}

Test case 4 :

Input:
{'Dev': [75, 75, 75], 'Simran': [49, 50, 51]}

Output:
{'Dev': (225, 75.0, 'A'), 'Simran': (150, 50.0, 'B')}

Test case 5 :

Input:
{'Anu': [100, 0, 100], 'Jay': [25, 25, 25]}

Output:
{'Anu': (200, 66.67, 'B'), 'Jay': (75, 25.0, 'C')}

Test case 6 :

Input:
{'Mira': [90, 80, 70], 'Kia': [55, 60, 65]}

Output:
{'Mira': (240, 80.0, 'A'), 'Kia': (180, 60.0, 'B')}

Q7. Arctic Research Ice-Core Thermal Analyzer

In a remote Arctic research base, scientists study thermal readings from ice-core extraction machines. These readings help determine melting patterns and hidden geothermal activity. But due to freezing winds, sensor fogging, and device malfunctions, some values become corrupted. Each ice-core machine records temperature readings in a tuple (integers or floats). However, the harsh environment often causes invalid entries like text or unreadable symbols. Your task is to clean these readings and figure out which machines show stable thermal output.

Your Task

Write a function **analyze_icecore_thermal(thermal_data: list) -> tuple** that:

- Accepts a list of tuples, each containing temperature readings for one machine.
- Example: `[(155, 140.5, 133), (250, "GLITCH", 240), (95, 100, 102)]`

For each tuple:

- Ignore any invalid (non-numeric) entries.
- Compute the average of only valid readings.
- If the average is **greater than 120**, the machine is considered **"stable."**
- Collect the rounded integer averages of all stable machines and return them as a tuple.

Exception Handling Rules (exactly 2)

Use try–except to handle:

- **TypeError** → if input is not a list → return `"Invalid Data Type"`
- **ValueError** → if the list is empty → return `"No Thermal Data Found"`

Input Format

```
A list of tuples containing numeric or mixed-type thermal readings
```

Output Format

```
A tuple of integers representing the rounded average thermal readings of stable machines,
 or one of the error messages:
 "Invalid Data Type"
 "No Thermal Data Found"
```

Sample Input 1

```
[(155, 140.5, 133), (250, "GLITCH", 240), (95, 100, 102)]
```

Sample Output 1

```
(143, 245)
```

Sample Input 2

```
[]
```

Sample Output 2

```
No Thermal Data Found
```

ANSWER:

Test case 1 :

Input:
[(155, 140.5, 133), (250, "GLITCH", 240), (95, 100, 102)]

Output:
(143, 245)

Test case 2 :

Input:
[]
Output:
No Readings Found

Test case 3 :

Input:
"notalist"
Output:
Invalid Data Type

Test case 4 :

Input:
[(120, 121, 122), (90, 80, 70)]
Output:
(121,)

Test case 5 :

Input:
[(200, 250, "??", 300), (50, 60, "oops")]
Output:
(250,)

Test case 6 :

Input:
[(100, "bad", 110), (118.5, 119.5), (200, 201)]
Output:
(200,)

Q8. Ancient Ruins Vibration Stability Detector

Archaeologists studying ancient ruins use vibration detectors to measure structural stability. But dust storms, collapsed walls, and unstable ground often corrupt some recorded vibration values. Each detector's vibration data is stored as a tuple of readings (integers or floats). Sometimes, faulty sensors capture invalid entries such as text or symbols. Your task is to clean these readings and determine which ruins exhibit stable vibration patterns.

Your Task

Write a function **detect_vibration_stability(vibes: list) -> tuple** that:

- Accepts a list of tuples, where each tuple contains vibration readings for a ruin section.
- Example: `[(160, 135.2, 128), (180, "NOISE", 190), (70, 85, 92)]`

For each tuple:

- Ignore any invalid (non-numeric) values.
- Compute the average of valid readings.
- If the average is **greater than 120**, the ruin section is considered **"stable."**
- Collect the rounded averages of all stable sections and return them as a tuple.

Exception Handling Rules (exactly 2)

Use try–except to handle:

- **TypeError** → if input is not a list → return `"Invalid Data Type"`
- **ValueError** → if the list is empty → return `"No Vibration Data Found"`

Input Format

```
A list of tuples containing numeric or mixed-type vibration readings
```

Output Format

```
A tuple of integers representing the rounded average vibration readings of stable ruin sections,
 or one of the error messages:
 "Invalid Data Type"
 "No Vibration Data Found"
```

Sample Input 1

```
[(160, 135.2, 128), (180, "NOISE", 190), (70, 85, 92)]
```

Sample Output 1

```
(141, 185)
```

Sample Input 2

```
12345
```

Sample Output 2

```
Invalid Data Type
```

ANSWER:

Test case 1 :

Input:
[(160, 135.2, 128), (180, "NOISE", 190), (70, 85, 92)]
Output:
(141, 185)

Test case 2 :

Input:
[]
Output:

No Readings Found

Test case 3 :

Input:
"notalist"
Output:
Invalid Data Type

Test case 4 :

Input:
[(120, 121, 122), (90, 80, 70)]
Output:
(121,)

Test case 5 :

Input:
[(200, 250, "??", 300), (50, 60, "oops")]
Output:
(250,)

Test case 6 :

Input:
[(100, "bad", 110), (118.5, 119.5), (200, 201)]
Output:
(200,)

Q9. Space Station Oxygen Module Stability Scanner

On an orbital research station, engineers constantly monitor oxygen module output levels to ensure the crew's safety. However, cosmic dust, micro-meteor impacts, and sensor glitches sometimes corrupt the readings. Each module's oxygen output is recorded as a tuple containing readings (integers or floats). Occasionally, the system mistakenly logs invalid values such as text or special characters. Your task is to filter corrupted values and determine which oxygen modules are functioning stably.

Your Task

Write a function **scan_oxygen_modules(modules: list) -> tuple** that:

- Receives a list of tuples, where each tuple stores oxygen output readings for one module.
- Example: `[(145, 138.4, 129), (260, "BAD", 255), (110, 115, 118)]`

For each tuple:

- Ignore all non-numeric values.
- Compute the average of valid readings.
- If the average is **greater than 120**, the module is labeled **"stable."**
- Gather the rounded averages of all stable modules and return them as a tuple.

Exception Handling Rules (exactly 2)

Use try–except to handle:

- **TypeError** → input is not a list → return `"Invalid Data Type"`
- **ValueError** → list is empty → return `"No Oxygen Data Found"`

Input Format

```
A list of tuples containing numeric or mixed-type oxygen output readings
```

Output Format

```
A tuple of integers representing the rounded average output of stable oxygen modules,
 or one of the error messages:
 "Invalid Data Type"
 "No Oxygen Data Found"
```

Sample Input 1

```
[(145, 138.4, 129), (260, "BAD", 255), (110, 115, 118)]
```

Sample Output 1

```
(137, 258)
```

Sample Input 2

```
[]
```

Sample Output 2

```
No Oxygen Data Found
```

ANSWER:

Test case 1 :

Input:
[(145, 138.4, 129), (260, "BAD", 255), (110, 115, 118)]
Output:
(137, 258)

Test case 2 :

Input:
[]
Output:
No Readings Found

Test case 3 :

Input:
"notalist"
Output:
Invalid Data Type

Test case 4 :

Input:
[(120, 121, 122), (90, 80, 70)]
Output:
(121,)

Test case 5 :

Input:
[(200, 250, "??", 300), (50, 60, "oops")]
Output:
(250,)

Test case 6 :

Input:
[(100, "bad", 110), (118.5, 119.5), (200, 201)]
Output:
(200,)